

UD 11. LECTURA Y ESCRITURA DE INFORMACIÓN

**Programación – 1º DAM
Curso 2025/2026**

Índice

- Ficheros en Java
- Operaciones con ficheros
- Ficheros de texto

Ficheros en Java

Java permite manejar archivos de texto y binarios mediante las bibliotecas **java.io** y **java.nio.file**. Las principales características y diferencias entre ficheros de texto y binarios son:

- **Ficheros de Texto:** sólo contienen caracteres, es decir, no tienen formato.
 - Formato legible por humanos.
 - Almacenados en ASCII o UTF-8.
 - Ejemplo: .txt, .csv.Ficheros
- **Binarios:** contienen datos de todo tipo, codificados byte a byte (binario).
 - No legibles directamente.
 - Usados para imágenes, audio, serialización, etc.
 - Ejemplo: .dat, .jpg, .mp3.

Operaciones con ficheros

Las principales operaciones que se realizan con ficheros son:

- Creación
- Apertura
- Lectura
- Escritura
- Recorrido
- Cierre

Ficheros de texto

- Un fichero de texto está formado por secuencias de caracteres, organizados en líneas de igual o distinta longitud.
- Todos los datos que aparecen en estos ficheros están formados por caracteres.
- Existen diferentes opciones de codificación en función del idioma usado (juegos de caracteres. Es usual el juego de caracteres UTF-8)
- Otra forma de nombrarlos es “archivos de texto llano/plano”, “texto simple” al no disponer de información sobre su formato y tipos de letra.
- Son legibles por las personas.

Ficheros de texto

Es habitual el uso de las siguientes clases (Para ficheros de texto plano):

- **File:** Se utiliza para comprobar si existe el fichero especificado
 - **FileWriter:** Para especificar archivo que se va a escribir
 - **BufferedWriter:** Escribir texto de manera eficiente en archivos.
 - **PrintWriter:** Para escribir texto con más funcionalidades (formato, auto-flush)
- Escritura en ficheros.** Hay 2 alternativas:
- FileWriter + BufferedWriter
 - FileWriter + PrintWriter
-
- **Scanner:** Para leer archivo (también sirve para consola)
 - **FileReader:** Para especificar el archivo del que se va a leer
 - **BufferedReader:** Para leer el archivo especificado
- Lectura de ficheros.** Hay 2 alternativas:
- Scanner
 - FileReader + BufferedReader

Ficheros de texto

Escritura en fichero con las siguientes clases: **FileWriter** y **PrintWriter**.

FILEWRITER

- La clase **FileWriter** permite tener acceso al fichero en modo escritura.
- Para crear objetos **FileWriter** podemos utilizar los constructores:

<code>FileWriter(String path)</code>	} El fichero se crea y si ya existe su contenido se pierde, se sobrescribe .
<code>FileWriter(File objetoFile)</code>	

- Si lo que necesitamos es abrir un fichero de texto existente sin perder su contenido y añadir más contenido al final utilizaremos los constructores:

<code>FileWriter(String path, boolean append)</code>	} Si append es true los datos se añaden a los existentes. Si es false los datos se pierden.
<code>FileWriter(File objetoFile, boolean append)</code>	

Ficheros de texto

- La clase **FileWriter** proporciona el método **write()** para escribir cadenas de caracteres, aunque lo normal es utilizar esta clase junto con la clase **PrintWriter** para facilitar la escritura.
- La clase **PrintWriter** permite escribir caracteres en el fichero de la misma forma que en la pantalla. Un objeto **PrintWriter** se crea a partir de un objeto **FileWriter**. **Ejemplo:**

```
FileWriter fw = new FileWriter("c:/docs/datos.txt");  
PrintWriter salida = new PrintWriter(fw);
```

En este ejemplo se ha creado un fichero de texto llamado **datos.txt**. El fichero se encuentra dentro de la carpeta ficheros en la unidad C:

Ficheros de texto

A partir de Java 5 se puede crear un objeto `PrintWriter` directamente a partir de un objeto `File` o de la ruta:

```
PrintWriter salida=new PrintWriter("c:/docs/datos.txt");
```

En este caso, si el fichero no existe se crea. Si no se puede crear un archivo con ese nombre o si ocurre algún error se lanza una excepción **`FileNotFoundException`**.

Ficheros de texto

Son **métodos de PrintWriter**:

- **print()**: se usa para escribir datos en forma de texto plano, sin agregar un salto de línea automáticamente.
- **println()**: escribe datos y agrega automáticamente un salto de línea (`\n`). Muy útil para escribir textos línea por línea.
- **printf()**: Permite escribir con formato, muy útil para texto alineado, decimales, etc. (`%d`, `%s`, `%.2f`, etc.). No agrega salto de línea automático (puedes usar `%n` si lo deseas).
- **flush()**: provoca que se escriban en el fichero los datos que pueda haber en el buffer de salida.
- **close()**: cierra la conexión con el fichero y libera los recursos en uso.

Ficheros de texto

Ejemplo de escritura:

```
public class Ejemplo1 {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        PrintWriter salida = null;  
        try {  
            salida = new PrintWriter("c:/docs/datos.txt"); // se crea el fichero  
            String cadena;  
            System.out.println("Introduce texto. Para acabar introduce la cadena FIN:");  
            cadena = sc.nextLine(); // se introduce por teclado una cadena de texto  
            while (!cadena.equalsIgnoreCase("FIN")) {  
                salida.println(cadena); // se escribe la cadena en el fichero  
                cadena = sc.nextLine(); // se introduce por teclado una cadena de texto  
            }  
            salida.flush();  
        } catch (FileNotFoundException e) {  
            System.out.println(e.getMessage());  
        } finally {  
            salida.close();  
        }  
    }  
}
```

Ficheros de texto

A partir de Java 7 se puede usar la instrucción **try-with-resources**. Un resource (recurso) es un objeto que necesita ser cerrado después de usarlo. Una instrucción **try-with-resources** asegura que estos objetos serán cerrados al final de la instrucción try.

A continuación del try se escribe el recurso entre paréntesis. Ahora no es necesario escribir el bloque finally para cerrar el fichero.

La instrucción try puede contener varios recursos, en este caso irán separados por punto y coma.

Ficheros de texto

El ejemplo anterior quedaría:

```
public class Ejercicio1ConRecursos {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
        String cadena;  
  
        try (PrintWriter salida = new PrintWriter("c:/docs/datos.txt")) { // try with resources  
            System.out.println("Introduce texto. Para acabar introduce la cadena FIN:");  
            cadena = sc.nextLine();  
            while (!cadena.equalsIgnoreCase("FIN")) {  
                salida.println(cadena);  
                cadena = sc.nextLine();  
            }  
        } catch (FileNotFoundException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Ficheros de texto

Lectura de fichero con las siguientes clases: **FileReader** y **BufferedReader**.

FILEREADER

- La clase **FileReader** permite tener acceso al fichero en modo lectura.
- Para crear objetos **FileReader** podemos utilizar los constructores:

```
FileReader(String path)
```

```
FileReader(File objetoFile)
```

Ambos lanzan una excepción **FileNotFoundException** si el fichero no existe.

La clase **FileReader** proporciona el método **read()** para leer caracteres del fichero, aunque lo normal es realizar la lectura mediante la clase **BufferedReader**.

Ficheros de texto

Para leer utilizando la clase **BufferedReader** se debe crear un objeto **BufferedReader** a partir de un objeto **FileReader**. **Ejemplo:**

```
FileReader fr = new FileReader("c:/docs/datos.txt");  
BufferedReader entrada = new BufferedReader(fr);
```

Una vez creado el objeto **BufferedReader** podemos usar los siguientes métodos:

- **readLine():** para leer líneas de texto del fichero (String). Este método devuelve null cuando no hay más líneas para leer.
- **read():** para leer carácter a carácter. Devuelve un entero que representa el código Unicode del carácter leído. Devuelve -1 si no hay más caracteres.

El fichero se debe cerrar cuando ya no se use, mediante el método **close()**.

Estos métodos lanzan una excepción **IOException** si ocurre un error de lectura.

Ficheros de texto

Ejemplo de lectura:

```
public class Ejemplo2_LecturaFicheros {  
    public static void main(String[] args) {  
  
        FileReader fr = null;  
  
        try {  
            fr = new FileReader("c:/docs/datos.txt");  
            BufferedReader entrada = new BufferedReader(fr);  
            String cadena = entrada.readLine(); // se lee la primera línea del fichero  
            while (cadena != null) { // mientras no se llegue al final del fichero  
                System.out.println(cadena); // se muestra por pantalla  
                cadena = entrada.readLine(); // se lee la siguiente línea del fichero  
            }  
        } catch (FileNotFoundException e) {  
            System.out.println(e.getMessage());  
        } catch (IOException e) {  
            System.out.println(e.getMessage());  
        } finally {  
            try {  
                if (fr != null) {  
                    fr.close();  
                }  
            } catch (IOException e) {  
                System.out.println(e.getMessage());  
            }  
        }  
    }  
}
```

Ficheros de texto

Ejemplo de lectura carácter a carácter:

```
public class Ejemplo2_LecturaFicherosCaracteres {  
  
    public static void main(String[] args) {  
  
        FileReader fr = null;  
        try {  
            fr = new FileReader("c:/ficheros/datos.txt");  
            BufferedReader entrada = new BufferedReader(fr);  
            int car = entrada.read(); // se lee el primer carácter del fichero  
            while (car != -1) { // mientras no se llegue al final del fichero  
                System.out.print((char) car); // se muestra por pantalla  
                car = entrada.read(); // se lee el siguiente carácter del fichero  
            }  
        } catch (FileNotFoundException e) {  
            System.out.println(e.getMessage());  
        } catch (IOException e) {  
            System.out.println(e.getMessage());  
        } finally {  
            try {  
                if (fr != null) {  
                    fr.close();  
                }  
            } catch (IOException e) {  
                System.out.println(e.getMessage());  
            }  
        }  
    }  
}
```